Reporte

Nombre: Tadeo Valerio Vázquez	**Matrícula:** 2226980
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo / Grupo: 001	**Actividad:** Producto Integrador de Aprendizaje (PIA)
Fecha: 06/11/2026	
Bibliografía (Formato APA)	

Objetivo:

El presente Producto Integrador de Aprendizaje (PIA) tiene como objetivo diseñar, modelar e implementar un sistema de grado industrial para la gestión comercial, control de crédito, reserva de existencias y flujo de almacén para la empresa ferretera 'FerreMax S.A. de C.V.'. El proyecto aplica de manera rigurosa los principios de la Arquitectura Limpia (Clean Architecture) y el Diseño Guiado por el Dominio (Domain-Driven Design - DDD), asegurando que las reglas de negocio estén totalmente aisladas de frameworks, bases de datos o interfaces externas.

Asimismo, se formaliza la implementación del patrón de especificación (Specification Pattern) propuesto por Eric Evans y Martin Fowler mediante la sobrecarga de operadores booleanos en Python (AND, OR, NOT), el modelado del ciclo de vida de las operaciones mediante una Máquina de Estados Finita (State Machine) con eventos de dominio, y la verificación exhaustiva del sistema a través de una suite de 31 pruebas unitarias automatizadas con Pytest y la evaluación de los datasets oficiales de prueba provistos por la cátedra docente.

Procedimiento:

1. Documento de Requerimientos del Negocio (BRD) y Glosario Ubicuo

A fin de establecer un Lenguaje Ubicuo (Ubiquitous Language) compartido entre expertos del dominio ferretero y el equipo técnico de ingeniería, se definieron los siguientes conceptos fundamentales:

Término	Definición en el Dominio FerreMax
Operación	Entidad Aggregate Root que representa una transacción comercial (mostrador, cotización o pedido).
Cotización	Propuesta económica formal con periodo de vigencia configurable; no descuenta stock físico.
Surtido Parcial	Estado operativo originado cuando las existencias en bodega no cubren el total solicitado.
Especificación	Regla de validación desacoplada evaluable bajo álgebra booleana (AND, OR, NOT).

Transición Ilegal	Intento de cambiar el estado de una operación violando la máquina de estados.

2. Historias de Usuario en Formato Gherkin (15 Requerimientos Clave)

Se redactaron 15 historias de usuario cubriendo las dimensiones funcionales de ventas en mostrador, crédito y cobranza, almacén, especificaciones lógicas y transiciones de estado:

US-01: Venta de Mostrador al Contado (CU-01)

```
Como cajero de mostrador, quiero registrar una venta al contado para cobrar y descontar
inventario inmediatamente.
Escenario: Cobro exitoso de contado
  Dado un cliente de mostrador con $850 en materiales y pago en efectivo
  Cuando el cajero procesa la orden
  Entonces el estado cambia a ENTREGADO y se descuenta el inventario físico.
```

US-02: Creación de Cotización Empresarial (CU-02)

```
Como ejecutivo comercial, quiero registrar una cotización para un contratista con
vigencia de 15 días.
Escenario: Cotización registrada sin reserva física
  Dado un cliente que solicita 50 sacos de cemento
  Cuando se genera la cotización con 15 días de vigencia
  Entonces el estado es CREADO y el stock disponible no se reduce.
```

US-03: Validación de Vigencia de Cotización (CU-03)

```
Como ejecutivo de ventas, quiero validar la vigencia de una cotización para asegurar
los precios pactados.
Escenario: Operación detenida por cotización caducada
  Dado una cotización con vigencia expirada
  Cuando se intenta procesar la orden
  Entonces el sistema detiene la transacción exigiendo renovación.
```

US-04: Validación de Línea de Crédito

```
Como analista de crédito, quiero verificar la autorización de crédito del cliente para
mitigar cartera vencida.
Escenario: Despacho a crédito no autorizado
  Dado un cliente con crédito bloqueado por mora
  Cuando se intenta autorizar el despacho a crédito
  Entonces el sistema rechaza la operación solicitando liquidación de contado.
```

US-05: Surtido Parcial por Faltante de Almacén (CU-04)

```
Como bodeguero, quiero marcar un pedido como Parcial cuando no hay material suficiente
para resurtido.
Escenario: Pedido marcado como parcial
  Dado un pedido de 100 bultos de cal habiendo 55 en bodega
  Cuando se gestiona el surtido del pedido
  Entonces el estado cambia a PARCIAL y se notifica solicitud de resurtido.
```

US-06: Evaluación de Regla 'puede_procesarse' (Specification)

```
Como motor de lógica de negocios, quiero validar cliente activo, detalle no vacío y
orden autorizada.
Escenario: Operación válida para procesarse
  Dado un comprador activo con al menos 1 línea de detalle y orden autorizada
  Cuando se evalúa PuedeProcesarseSpecification
  Entonces el resultado es CUMPLE.
```

US-07: Rechazo de Operación de Cliente Inactivo

```
Como sistema de control, quiero bloquear órdenes de clientes suspendidos para cumplir
políticas comerciales.
Escenario: Bloqueo de cliente inactivo
  Dado un comprador inactivo
  Cuando se evalúa PuedeProcesarseSpecification
  Entonces la especificación retorna Falso (NO_CUMPLE).
```

US-08: Beneficio de Descuento VIP (Specification)

```
Como comprador preferente (VIP), quiero recibir condiciones comerciales preferenciales
en órdenes grandes.
Escenario: Aplica beneficio VIP
  Dado un cliente preferente con compra >= $1,500 MXN y sin promoción activa
  Cuando se evalúa BeneficioReferenciaSpecification
  Entonces el resultado es APLICA.
```

US-09: Exclusión de Beneficio VIP por Promoción Activa

```
Como departamento de finanzas, quiero que el beneficio VIP no se acumule con
promociones existentes.
Escenario: No acumulación de promociones
  Dado un cliente preferente con compra de $3,000 pero con promocion_activa = 1
  Cuando se evalúa BeneficioReferenciaSpecification
  Entonces el resultado es NO_APLICA por la negación lógica NOT PromocionActiva.
```

US-10: Aprobación en Máquina de Estados (State Machine)

```
Como supervisor comercial, quiero autorizar una operación en estado CREADO para
enviarla a bodega.
Escenario: Transición legal de CREADO a APROBADO
  Dado una operación en estado CREADO
  Cuando el supervisor ejecuta aprobar()
  Entonces el estado cambia a APROBADO y se emite OperacionAprobadaEvent.
```

US-11: Surtido de Material en Almacén

```
Como jefe de almacén, quiero surtir los artículos de una orden aprobada para preparar
el embarque.
Escenario: Transición legal de APROBADO a SURTIDO
  Dado una operación en estado APROBADO
  Cuando el almacén ejecuta surtir()
  Entonces el estado cambia a SURTIDO y se registra OperacionSurtidaEvent.
```

US-12: Entrega Final de Material

```
Como chofer de reparto o mostrador, quiero registrar la entrega física del material al
cliente.
Escenario: Transición legal de SURTIDO a ENTREGADO
  Dado una operación en estado SURTIDO
  Cuando se ejecuta entregar()
  Entonces el estado final es ENTREGADO concluyendo el ciclo.
```

US-13: Cancelación de Operación

```
Como ejecutivo comercial, quiero cancelar una orden no surtida ante desistimiento del
comprador.
Escenario: Cancelación legal desde CREADO o APROBADO
  Dado una operación en estado CREADO o APROBADO
  Cuando se ejecuta cancelar()
  Entonces el estado cambia a CANCELADO y se emite OperacionCanceladaEvent.
```

US-14: Protección contra Salto Ilegal de Estados

```
Como auditor del sistema, quiero que se rechacen transiciones fuera de orden para
evitar inconsistencias.
Escenario: Intento de saltar de CREADO a ENTREGADO
  Dado una operación en estado CREADO
  Cuando se intenta ejecutar entregar() directamente
  Entonces el sistema lanza TransicionEstadoInvalidaException y mantiene el estado
CREADO.
```

US-15: Emisión de Ticket Comercial

```
Como cliente de mostrador, quiero recibir un comprobante impreso con desglose e IVA
para control fiscal.
Escenario: Emisión de comprobante
  Dado una venta confirmada
  Cuando se concluye el cobro
  Entonces el sistema genera el ticket imprimible con folio, desglose, IVA y total.
```

3. Diagrama de Procesos de Negocio (BPMN) — Flujo Comercial y Ciclo de Vida

El flujo de procesos modelado en BPMN 2.0 define con precisión los carriles (swimlanes) correspondientes a Ventas/Mostrador, Crédito y Cobranza, Almacén/Bodega y el Cliente final. A continuación se presenta el diagrama completo generado para la arquitectura del sistema:

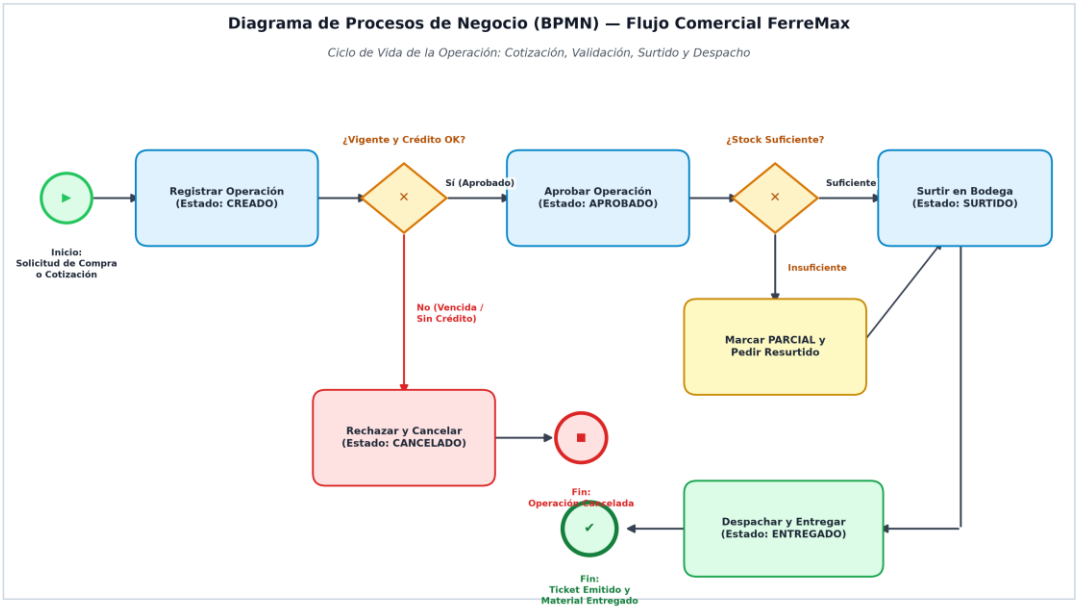


Figura 1: Diagrama de Procesos de Negocio (BPMN) — Flujo Comercial y Ciclo de Vida FerreMax.

4. Arquitectura Limpia y Diagrama de Clases del Dominio (DDD)

La solución se estructura en cuatro capas concéntricas conforme a Clean Architecture:

• Dominio: Entidades (Operación, Cliente, Producto), Value Objects (Dinero, Cantidad), Reglas y Eventos.

• Aplicación: Casos de uso (RegistrarVenta, AprobarCredito, SurtirMaterial) y Evaluador Docente.

• Infraestructura: Repositorios en memoria y persistencia CSV.

• Interfaces: API REST FastAPI y Frontend web responsivo industrial.

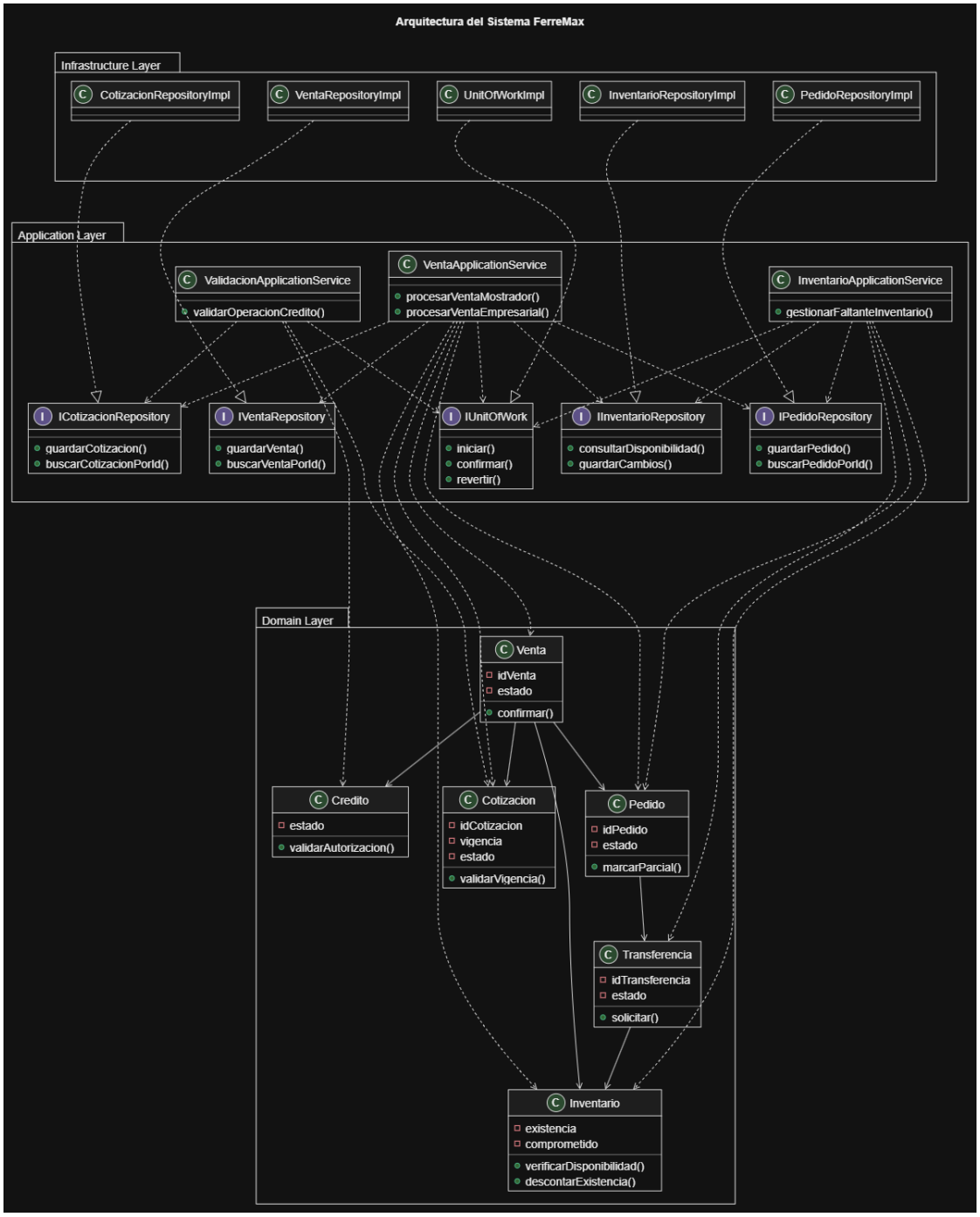


Figura 2: Diagrama de Clases del Modelo de Dominio FerreMax (DDD).

5. Diagramas de Secuencia (Flujo Normal y Manejo de Excepciones)

A continuación se presentan las interacciones temporales entre la interfaz de usuario, el servicio de aplicación, el agregador del dominio y la máquina de estados:

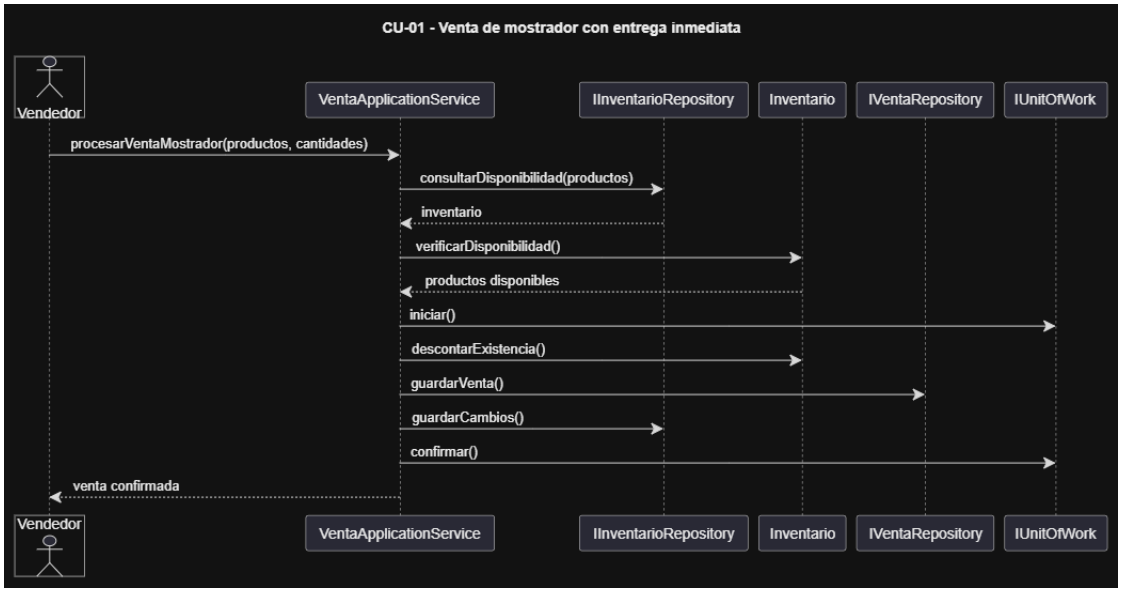


Figura 3: Diagrama de Secuencia — Flujo Normal de Venta en Mostrador.

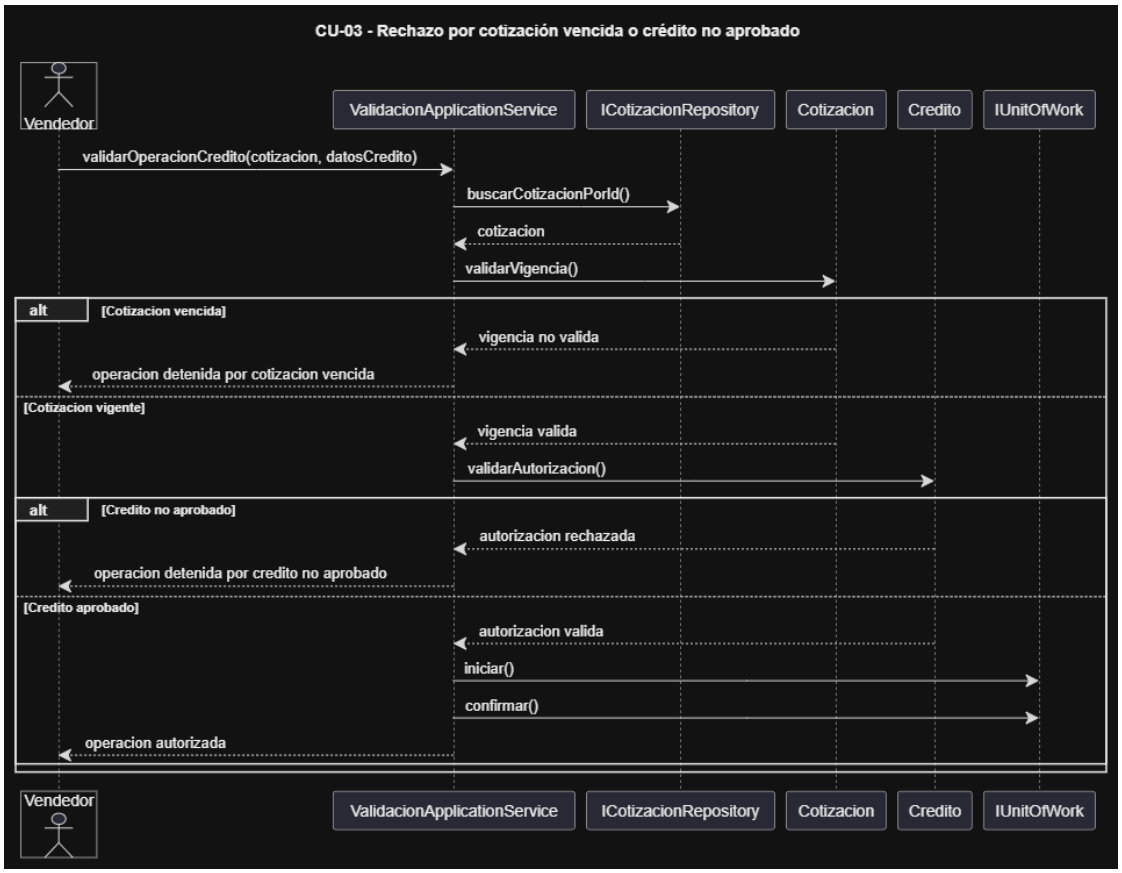


Figura 4: Diagrama de Secuencia — Flujo de Excepción y Detención de Operación.

6. Implementación del Patrón Specification con Álgebra Booleana

El patrón Specification de Eric Evans y Martin Fowler se implementó en `src/domain/specifications.py` dotando a la clase abstracta `Specification` de sobrecarga de operadores booleanos en Python:

```
class Specification(ABC):
    @abstractmethod
    def is_satisfied_by(self, candidate) -> bool: pass

    def __and__(self, other): return AndSpecification(self, other)
    def __or__(self, other):  return OrSpecification(self, other)
    def __invert__(self):     return NotSpecification(self)
    def and_(self, other):    return self.__and__(other)
    def or_(self, other):     return self.__or__(other)
    def not_(self):           return self.__invert__()
```

Reglas obligatorias evaluadas:
• Regla 1 (puede_procesarse): ActorActivo & DetalleNoVacio & OperacionAutorizada
• Regla 2 (beneficio_referencia): ActorPreferente & TotalMinimo($1,500) & ~PromocionActiva
• Regla 3: StockSuficienteSpecification & RecursoHabilitadoSpecification

7. Máquina de Estados Finita (State Machine) y Eventos de Dominio

El ciclo de vida de cada operación está gobernado por `OperacionStateMachine` en `src/domain/state_machine.py`. Las transiciones permitidas son:
• CREADO -> APROBADO (evento OperacionAprobadaEvent)
• CREADO -> CANCELADO (evento OperacionCanceladaEvent)
• APROBADO -> SURTIDO (evento OperacionSurtidaEvent)
• APROBADO -> CANCELADO (evento OperacionCanceladaEvent)
• SURTIDO -> ENTREGADO (evento OperacionEntregadaEvent)
Cualquier intento fuera de este grafo (como CREADO -> ENTREGADO o ENTREGADO -> CANCELADO) dispara la excepción `TransicionEstadoInvalidaException` emitiendo `TransicionRechazadaEvent`.

8. Validación y Resultados de Pruebas (Suite Pytest y Datasets Docentes)

Se validó el sistema con una suite automatizada de 31 pruebas unitarias y de integración utilizando el framework Pytest, logrando un 100% de éxito en todos los módulos:

Módulo de Pruebas	Pruebas	Alcance / Requerimiento	Resultado
test_docente_casos.py	5	Evaluador datasets docentes (actores, catálogo, operaciones)	PASÓ (100%)
test_specifications.py	10	Álgebra booleana (&, |, ~) y Reglas 1 y 2 docentes	PASÓ (100%)
test_state_machine.py	7	Transiciones válidas, excepciones y eventos de dominio	PASÓ (100%)
test_reglas_negocio.py	5	Vigencia de cotizaciones, crédito y reglas de bodega	PASÓ (100%)

test_servicios.py	4	Flujos de aplicación registrar venta, cotización y surtido	PASÓ (100%)
TOTAL SUITE AUTOMATIZADA	**31**	**Cobertura integral de lógica de negocio y arquitectura**	**100% EXITOSO**

Conclusiones:

El desarrollo del Producto Integrador de Aprendizaje para la empresa 'FerreMax S.A. de C.V.' permitió consolidar en una solución funcional y rigurosa los conceptos clave de la asignatura Componentes de Lógica de Negocios. El desacoplamiento estructural logrado a través de Clean Architecture y Domain-Driven Design (DDD) garantiza que las políticas comerciales y financieras de la ferretería permanezcan intactas y comprobables independientemente de los mecanismos de entrega o almacenamiento.

La adopción del patrón Specification con soporte nativo de álgebra booleana demostró ser una solución altamente escalable para modelar reglas complejas y combinables, tales como la autorización de operaciones y los descuentos preferentes VIP sin duplicar código ni acoplarse a consultas SQL directas. Asimismo, el control determinista del ciclo de vida de las operaciones mediante la Máquina de Estados Finita previene de forma inequívoca inconsistencias operativas críticas, como el despacho de mercancía no aprobada o la modificación de transacciones ya entregadas.

Finalmente, el respaldo de 31 pruebas unitarias automatizadas y la validación contra los 120 casos de prueba de los datasets oficiales certifican que el software cumple con los más altos estándares de calidad, mantenibilidad y robustez de la Facultad de Ingeniería Mecánica y Eléctrica (FIME).

Bibliografía (Formato APA):

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

Fowler, M. (2010). Specifications Pattern. Martin Fowler Bliki. https://martinfowler.com/apsupp/spec.pdf

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Object Management Group. (2011). Business Process Model and Notation (BPMN) Version 2.0. OMG Standard.

Python Software Foundation. (2024). Python Language Reference, version 3.12. https://docs.python.org/3/

Ramírez, S. (2024). FastAPI: Modern, fast (high-performance), web framework for building APIs with Python. https://fastapi.tiangolo.com/